

IMPLEMENTATION PROTOCOL — TRADING SUPER APP

PURPOSE

This document defines how you must operate when making changes to the Trading Super App.

The MASTER PROJECT SPECIFICATION defines the product vision.

This document defines the engineering workflow.

You must follow this protocol for every meaningful development task.

1. CORE RULE

DO NOT optimize for:

"How much code can I generate?"

Optimize for:

"How can I safely add this functionality to the existing system without breaking anything?"

The application is a growing production-style trading platform.

Every change must preserve the integrity of the existing application.

2. NEVER START BY CODING

When given a development request, do not immediately edit files.

First determine:

- what the user wants
- what currently exists
- where the relevant functionality lives
- what dependencies exist
- how data currently flows
- what existing components can be reused

- what could be affected

Inspect the relevant code before deciding how to implement the feature.

3. TASK CLASSIFICATION

Classify each task as one of:

TYPE A — UI ONLY

Example:

Change button appearance.

TYPE B — UI + EXISTING LOGIC

Example:

Add an existing metric to a dashboard.

TYPE C — NEW BUSINESS LOGIC

Example:

Add position-size calculation.

TYPE D — DATA INTEGRATION

Example:

Connect a live price stream.

TYPE E — BROKER FUNCTIONALITY

Example:

Add order modification.

TYPE F — ARCHITECTURAL CHANGE

Example:

Replace market-data architecture.

TYPE G — BUG FIX

Example:

Chart stops updating after reconnect.

TYPE H — SECURITY-CRITICAL

Example:

Change broker credential handling.

The more dangerous the category, the more investigation and testing is required.

4. BEFORE IMPLEMENTATION

For each meaningful task, produce a short implementation plan.

Use:

TASK

What is being implemented.

CURRENT STATE

What exists now.

TARGET STATE

What should exist after implementation.

FILES / MODULES

Which areas will be changed.

DATA FLOW

How information will move through the system.

RISKS

What could break.

TEST PLAN

How the result will be verified.

Then implement.

For trivial UI changes, this can be extremely brief.

5. SEARCH THE CODEBASE FIRST

Before creating a new:

- component
- hook
- service
- API route
- database model
- utility
- state store
- type
- broker adapter

search the repository for an existing implementation.

Never assume something doesn't exist.

6. REUSE BEFORE CREATING

Priority:

1. Existing implementation
2. Existing reusable component
3. Existing service
4. Existing utility
5. Refactor existing code
6. Create new code only when necessary

Avoid duplicate functionality.

7. DO NOT CREATE PARALLEL SYSTEMS

Never accidentally create:

two chart engines

two market-data services

two order services

two authentication systems

two analytics engines

two versions of the same component

two state-management systems

unless there is a deliberate architectural reason.

If an existing system already performs the required task, extend it.

8. DATA FLOW FIRST

For features involving data, define:

SOURCE ↓ SERVICE ↓ TRANSFORMATION ↓ STATE ↓ UI

For example:

Capital.com ↓ Capital Broker Adapter ↓ Normalized Market Data ↓ Market Data Store ↓ Chart / Watchlist / Order UI

Do not make each component communicate directly with the broker.

9. BROKER RULES

Existing Capital.com integration is valuable working functionality.

Existing MT5 integration is valuable working functionality.

DO NOT rewrite either casually.

Before changing broker code:

- understand authentication
- understand connection lifecycle
- understand API calls
- understand data structures
- understand error handling
- understand existing normalization
- understand how the UI currently consumes the data

Preserve compatibility.

10. MT5 LOCAL PROCESS

MT5 currently uses a local process running on the user's PC to obtain data.

Do not assume:

Frontend ↓ Cloud server ↓ Direct MT5 connection

is possible.

First understand the existing local bridge.

Document:

PC process ↓ MT5 ↓ data retrieval ↓ local service / bridge ↓ application

Then determine how it currently communicates with the application.

Do not replace this architecture without approval.

11. MARKET DATA RULE

All chart and market functionality should use real normalized market data.

Never:

- hardcode candles
- invent prices
- generate random price movements
- silently use demo data
- mix symbols
- mix timeframes

If mock data is required for development, isolate it behind an explicit simulation mode.

12. REAL-TIME SYSTEM RULE

For real-time features:

Initial state:

Historical API ↓ Chart

Then:

Live stream ↓ incremental update ↓ existing candle or new candle ↓ chart

Do not repeatedly download the entire historical dataset every time a tick arrives.

13. TIMEFRAME RULE

Always know the timeframe represented by the data.

A candle must have:

- timestamp
- open
- high
- low
- close
- volume if available

Do not mix timeframe data.

If aggregating candles:

Define the exact aggregation rules.

Test candle boundaries.

Be careful about:

- timezone
 - UTC
 - broker server time
 - daylight saving where applicable
 - session boundaries
-

14. CHART DEVELOPMENT RULE

The chart must be treated as a high-performance subsystem.

Do not recreate the chart instance unnecessarily.

Do not reload all candles on every price update.

Do not unnecessarily rerender the entire application on every tick.

Use incremental updates where the selected charting technology supports them.

15. CHART INTERACTION RULE

Any chart interaction must remain synchronized with:

- zoom
- pan
- resize
- timeframe
- symbol
- data updates

Drawing tools must use chart coordinates rather than arbitrary screen coordinates.

16. ORDER SYSTEM RULE

Never treat:

"Order button clicked"

as equivalent to:

"Order successfully executed."

Correct flow:

User ↓ Order form ↓ Validation ↓ Backend ↓ Broker adapter ↓ Broker ↓ Broker confirmation ↓
Normalized order state ↓ UI update

If broker rejects:

Broker rejection ↓ Error state ↓ User notification ↓ No false success

17. DUPLICATE ORDER PROTECTION

Prevent accidental repeated order submission.

Consider:

- disabled submit during request
- unique request IDs
- idempotency where supported
- broker order identifiers
- timeout handling

A network retry must not accidentally create two orders.

18. ORDER STATE

Orders should have meaningful states.

Examples:

PENDING SUBMITTING ACCEPTED REJECTED PARTIALLY_FILLED FILLED CANCEL_REQUESTED CANCELLED
MODIFICATION_PENDING MODIFIED ERROR

Use actual broker state whenever possible.

Do not invent success states.

19. POSITION STATE

Positions should reflect actual broker state.

Do not calculate a fake position merely because the user clicked BUY.

A position should be confirmed through the appropriate broker response/state.

20. UI STATE VS SERVER STATE

Do not confuse:

"User requested an order"

with:

"Broker confirmed an order."

The frontend can temporarily display:

Submitting...

but must eventually resolve to actual server/broker state.

21. ERROR RECOVERY

Every network feature must consider:

- timeout
- connection failure
- server error
- broker error
- invalid response
- authentication expiration
- rate limit
- WebSocket disconnect

Do not assume requests always succeed.

22. WEBSOCKET RECONNECTION

When a real-time connection fails:

1. mark connection as disconnected/reconnecting
2. stop treating old data as current
3. reconnect using controlled backoff
4. reauthenticate/resubscribe if necessary
5. synchronize missing data
6. resume live updates
7. verify state consistency

Do not simply reconnect in a tight infinite loop.

23. STALE DATA

Track the last received update.

If market data becomes stale:

show an appropriate state.

For example:

LIVE

RECONNECTING

STALE

DISCONNECTED

The exact threshold depends on the data source.

24. API ERROR HANDLING

Every API layer should have consistent error handling.

Do not expose raw technical errors directly to users.

Convert errors into:

- useful user message
- internal diagnostic information

Example:

User:

"Order could not be submitted."

Internal:

Broker rejected order: INVALID_VOLUME

25. DATABASE CHANGES

Before changing the database:

Inspect:

- existing schema
- relations
- migrations
- indexes
- constraints
- existing data

Never casually rename or delete production fields.

If migration is required:

- explain it
 - make it reversible where practical
 - avoid destructive migrations without explicit approval
-

26. DATABASE PERFORMANCE

Consider indexes for frequently queried fields such as:

- user ID
- account ID
- symbol

- timestamp
- order ID
- position ID

Do not create indexes blindly.

27. ANALYTICS RULE

Analytics must use actual recorded trades.

Never generate performance numbers from UI state alone.

Analytics should have a clear source of truth.

For example:

Broker/Trade records ↓ Trade normalization ↓ Trade database ↓ Analytics engine ↓ Metrics ↓ Dashboard

28. ANALYTICS CONSISTENCY

Metrics must use clearly defined formulas.

For example:

Win Rate

Winning Trades / Total Closed Trades

Profit Factor

Gross Profit / Absolute Gross Loss

Expectancy

Average Win × Win Rate

- Average Loss × Loss Rate

The exact project definitions should be documented.

Do not silently change metric definitions.

29. AI ANALYSIS PIPELINE

AI analysis should NOT directly query random sources itself without application control.

Preferred architecture:

User selects symbol ↓ Market Data Service ↓ Technical Analysis Engine ↓ News Service ↓ Relevant Context ↓ Structured AI Input ↓ AI Model ↓ Structured Analysis ↓ UI

30. AI INPUT VALIDATION

Before sending information to the AI:

Verify:

- symbol
- timestamp
- current price
- timeframe
- indicator values
- news timestamps
- news sources

The AI should know the timestamp of the data it is analyzing.

31. AI OUTPUT STRUCTURE

Prefer structured AI responses.

Example conceptual structure:

```
{ market_summary, trend, technical_analysis, key_levels, momentum, volatility, news, bullish_factors, bearish_factors, risks, scenarios, invalidation, confidence, sources }
```

The exact implementation should match the project's technology.

Avoid relying exclusively on free-form text if the UI needs to display individual analysis sections.

32. AI SOURCE TRACEABILITY

When AI uses external information:

Store or retain appropriate source metadata.

At minimum where applicable:

- source
- headline
- URL
- timestamp
- relevance

Do not fabricate citations.

33. AI FAILURE HANDLING

If the AI service fails:

The application should still work.

For example:

"AI analysis temporarily unavailable."

The chart, market data, and trading functions must not depend on AI availability.

34. AI MUST NOT CONTROL TRADING

AI analysis must remain separate from execution.

Never automatically execute:

BUY SELL CLOSE MODIFY

based solely on AI output.

35. SECURITY CHECK BEFORE CODE

When touching:

- authentication
- broker credentials
- API keys
- user accounts
- payments
- trading permissions
- server configuration

perform a security review before implementation.

Never expose secrets.

36. FRONTEND COMPONENT RULE

Components should have clear responsibilities.

Avoid giant components containing:

- API calls
- trading calculations
- WebSocket management
- database logic
- UI rendering
- business logic

Split responsibilities appropriately.

37. UI COMPONENT RULE

Before creating a component, search for similar components.

If one exists:

reuse or extend it.

If several similar components exist:

consider consolidating them if safe.

38. DESIGN SYSTEM RULE

Do not invent a new visual language for every screen.

Use existing:

- spacing
- typography
- colors
- buttons
- inputs
- panels
- tables
- dialogs
- navigation

If the current design system is weak, improve the design system systematically rather than fixing every page individually.

39. AI-GENERATED UI AVOIDANCE

Do not automatically produce:

- giant rounded cards
- gradients everywhere
- excessive glass effects
- oversized headings
- random icons
- meaningless statistics
- excessive whitespace
- decorative dashboards

The interface should prioritize information density and usability appropriate for a trading application.

40. RESPONSIVE RULE

Never consider a desktop page "mobile complete" simply because it technically fits the screen.

For every major screen ask:

"What does a trader actually need on a phone?"

Then redesign interaction appropriately.

41. MOBILE ORDER SAFETY

For mobile trading:

- make order details highly visible
 - make BUY/SELL distinct
 - show quantity
 - show SL
 - show TP
 - show risk
 - prevent accidental double submission
 - use confirmation where appropriate
-

42. ACCESSIBILITY

Do not rely only on:

- color
- hover
- icons

Use appropriate labels and semantic structure.

43. PERFORMANCE RULE

Before optimizing:

measure or identify an actual bottleneck.

Do not blindly add:

- memoization everywhere
- caching everywhere
- complex state systems
- workers
- microservices

Optimization should solve real problems.

44. DEPENDENCY RULE

Before adding a package:

Check:

- Does the project already have a solution?
 - Is this dependency necessary?
 - Is it maintained?
 - Is its license appropriate?
 - Does it increase bundle size?
 - Does it create long-term coupling?
-

45. NO UNNECESSARY REWRITES

Never rewrite an entire module because:

"the existing code isn't clean."

Refactor incrementally unless the current implementation prevents the required functionality.

46. NO UNRELATED CHANGES

If implementing:

"Add RSI"

do not simultaneously redesign:

- homepage
- authentication
- database
- navigation
- broker system

unless required.

Keep the change scoped.

47. GIT SAFETY

Before major changes:

Understand the current working state.

Do not delete unrelated work.

Do not overwrite user changes.

Do not reset/revert files unless explicitly instructed.

Keep changes logically grouped.

48. TESTING REQUIREMENT

Testing must happen at multiple levels where appropriate.

Unit

Business logic.

Integration

API/service communication.

End-to-end

Important user workflows.

Manual

Visual and interaction verification.

49. TRADING WORKFLOW TEST

For major trading functionality, verify:

1. Login
2. Account connection
3. Market data

4. Symbol selection
 5. Chart
 6. Order form
 7. Validation
 8. Order submission
 9. Broker response
 10. Position update
 11. P&L update
 12. Order history
 13. Refresh/reconnect behavior
-

50. FAILURE TESTING

Do not only test success.

Test:

- invalid symbol
 - invalid quantity
 - insufficient margin
 - disconnected broker
 - expired authentication
 - rejected order
 - network timeout
 - WebSocket disconnect
 - empty history
 - unavailable news
 - AI failure
-

51. VISUAL QA

After UI implementation inspect:

- desktop
- tablet where applicable
- mobile
- dark mode if supported
- long text
- empty state
- loading state
- error state
- large datasets

Check:

- alignment
 - spacing
 - overflow
 - clipping
 - responsiveness
 - typography
 - consistency
-

52. DEFINITION OF DONE

Before saying:

"Complete"

verify:

☐ Feature implemented ☐ Existing functionality preserved ☐ Data flow verified ☐ Error handling implemented ☐ Loading state implemented ☐ Empty state implemented where relevant ☐ Security considered ☐ Tests performed ☐ Mobile behavior checked where relevant ☐ No unnecessary dependencies ☐ No fake production data ☐ Documentation updated where necessary

53. COMPLETION REPORT

After implementation provide:

SUMMARY

What was done.

FILES CHANGED

List relevant files.

ARCHITECTURE

Explain important data flow changes.

TESTS

Explain exactly what was tested.

RESULTS

What passed.

LIMITATIONS

What remains.

NEXT STEP

Recommend the next logical task.

Never claim tests passed if they were not actually run.

54. IF SOMETHING IS UNCLEAR

Do not guess when guessing could affect:

- financial behavior
- security
- database structure
- broker execution
- user data
- architecture

Ask for clarification.

For minor UI decisions, use the existing design system and established product principles.

55. IF YOU DISCOVER A BUG

Do not immediately patch symptoms.

Determine:

1. Root cause
2. Affected systems

3. Why it happened
4. Correct fix
5. Regression risk
6. Test required

Fix the root cause where practical.

56. IF THE EXISTING ARCHITECTURE IS BAD

Do not silently replace it.

Explain:

CURRENT PROBLEM

WHY IT MATTERS

PROPOSED ARCHITECTURE

MIGRATION STRATEGY

RISKS

BENEFITS

Then get approval for major architectural changes.

57. IF THE REQUEST IS TOO LARGE

Break it into phases.

Example:

"Build TradingView-style chart"

should become:

1. Chart architecture
2. Historical candles
3. Real-time candles
4. Timeframes
5. Crosshair

6. Zoom/pan
7. Indicators
8. Drawing tools
9. Order visualization
10. Persistence
11. Performance optimization

Do not attempt all ten simultaneously unless explicitly instructed.

58. VERTICAL SLICE PRINCIPLE

When possible, build complete vertical slices.

Example:

Instead of:

"Build all frontend screens first."

Build:

Symbol selection ↓ Real market data ↓ Chart ↓ One working timeframe ↓ One working live update ↓
Testing

Then expand.

A small fully working system is more valuable than a large disconnected UI.

59. INCREMENTAL CHART DEVELOPMENT

Recommended sequence:

Step 1

Historical candlestick chart.

Step 2

Multiple timeframes.

Step 3

Symbol switching.

Step 4

Real-time price updates.

Step 5

Live candle updates.

Step 6

Crosshair.

Step 7

Zoom/pan.

Step 8

Volume.

Step 9

Indicators.

Step 10

Drawing tools.

Step 11

Order/position visualization.

Step 12

Persistence.

60. INCREMENTAL TRADING DEVELOPMENT

Recommended sequence:

Step 1

Display account.

Step 2

Display real positions.

Step 3

Display orders.

Step 4

Market order in paper/demo mode.

Step 5

Real broker order submission.

Step 6

Order confirmation.

Step 7

Position updates.

Step 8

SL/TP.

Step 9

Order modification.

Step 10

Chart order interaction.

61. INCREMENTAL AI DEVELOPMENT

Recommended sequence:

Step 1

Collect real market data.

Step 2

Calculate technical indicators.

Step 3

Collect relevant news.

Step 4

Normalize information.

Step 5

Send structured context to AI.

Step 6

Return structured analysis.

Step 7

Display analysis.

Step 8

Display sources.

Step 9

Add historical analysis tracking.

Step 10

Evaluate analysis quality.

62. PROJECT MEMORY

Maintain a living document such as:

PROJECT_STATE.md

It should contain:

Current architecture

Working features

In-progress features

Known bugs

Broker integrations

Market data providers

Database schema summary

Current chart capabilities

Current AI capabilities

Design system

Technical debt

Next priorities

Update it when major architecture or feature changes occur.

This prevents the AI from repeatedly forgetting the state of the project.

63. FEATURE STATUS

Use clear statuses:

NOT STARTED

PLANNED

IN PROGRESS

IMPLEMENTED

TESTING

VERIFIED

BLOCKED

DEPRECATED

Do not mark a feature VERIFIED without testing.

64. NEVER HIDE TECHNICAL DEBT

If a quick implementation introduces technical debt, document it.

Do not pretend the temporary solution is production-ready.

65. NO FAKE FALLBACKS

This is especially important.

Do not do:

try real API catch → show fake price

unless the application is explicitly in simulation mode.

A failure should be visible as a failure.

66. TRADING DATA INTEGRITY

Never mix:

- demo account data
- live account data
- simulated data
- historical data
- stale data

without clearly identifying the source/state.

The user must always know whether they are looking at:

LIVE DEMO PAPER HISTORICAL SIMULATED

67. LIVE TRADING ENVIRONMENT

Live trading should have clear environment separation.

Never allow development code to accidentally send live orders.

Before enabling live execution:

verify environment and account context.

68. AI ANALYSIS DATA FRESHNESS

AI analysis should show when the underlying data was collected.

Example:

Analysis generated: 13:42

Market data: 13:41

News checked: 13:38

This prevents users from assuming analysis is based on continuously current information when it is not.

69. MARKET STATUS

Where supported, distinguish:

OPEN

CLOSED

PRE-MARKET

AFTER-HOURS

HALTED

UNAVAILABLE

Do not assume all markets operate 24/7.

70. TIMEZONE

Internally prefer a consistent timezone strategy, typically UTC for stored timestamps unless the existing architecture requires otherwise.

Convert to local/user timezone only at the presentation layer where appropriate.

Never mix timestamps without knowing their timezone.

71. FINANCIAL PRECISION

Do not rely blindly on JavaScript floating-point arithmetic for financial calculations.

Use appropriate decimal/precision handling where required.

Respect:

- tick size
- decimal precision
- lot size
- minimum quantity
- minimum distance
- contract specifications

These are broker/instrument-specific.

72. SYMBOL METADATA

Do not assume every instrument has:

- same tick size
- same decimal places
- same lot size
- same leverage

- same trading hours
- same contract size

Use instrument metadata where available.

73. ORDER VALIDATION

Order validation must account for instrument-specific rules.

Examples:

- minimum volume
 - volume step
 - minimum stop distance
 - price precision
 - trading session
 - margin requirement
-

74. FINANCIAL UI PRECISION

Do not unnecessarily round values internally.

Round only for presentation where appropriate.

The displayed number and underlying calculation should be distinguishable.

75. USER EXPERIENCE PRINCIPLE

Every major action should answer:

"What just happened?"

For example:

Order submitted → submitting

Broker confirms → filled

Broker rejects → rejected + reason

Connection lost → reconnecting

AI unavailable → clearly unavailable

Never leave users guessing.

76. FINAL RULE

When uncertain, choose:

REAL FUNCTIONALITY over VISUAL SIMULATION

SIMPLE ARCHITECTURE over UNNECESSARY COMPLEXITY

EXISTING WORKING CODE over REWRITING

TESTED CODE over ASSUMED CODE

CLEAR FAILURE over FAKE SUCCESS

USER SAFETY over CONVENIENCE

PRODUCT QUALITY over CODE GENERATION SPEED

END OF IMPLEMENTATION PROTOCOL